



IA-DataFlow-Hub — Arquitectura del Proyecto

Info

Plataforma monorepo para automatización de flujos de datos e integración de IA local usando **React + NestJS + Docker + n8n + LM Studio**.



Índice

1. Visión General
2. Estructura del Monorepo
3. Stack Tecnológico
4. Servicios Docker
5. Frontend – apps/client
6. Backend – apps/api
7. Base de Datos – packages/database
8. Tipos Compartidos – packages/shared-types
9. Automatización – n8n
10. IA Local – ai-services
11. Infraestructura – infra/nginx
12. Variables de Entorno
13. Flujo de Desarrollo
14. Flujo de Producción

Visión General

Descripción

IA-DataFlow-Hub es una plataforma de gestión y automatización de flujos de datos con integración de inteligencia artificial local.

La arquitectura está basada en:

-  Monorepo con Turborepo
-  Docker Compose
-  React + Vite

-  NestJS
-  MySQL + Prisma
-  LM Studio
-  n8n

Accesos Locales

Servicio	URL
Frontend	<code>http://localhost:5173</code>
API REST	<code>http://localhost:3000</code>
n8n	<code>http://localhost:5678</code>
phpMyAdmin	<code>http://localhost:8080</code>
MySQL externo	<code>localhost:3307</code>

Arquitectura General

```
Usuario
├── Frontend React (5173)
├── API NestJS (3000)
├── n8n Automation (5678)
├── phpMyAdmin (8080)
└── MySQL (3307)
```

Tip

Todos los servicios corren dentro de la red Docker interna:

```
iadataflow_net
```

2 Estructura del Monorepo

📁 Organización General

```
IA-DataFlow-Hub/
|
├── apps/
|   ├── client/           # Frontend React + Vite
|   └── api/              # Backend NestJS
|
├── packages/
|   ├── database/         # Prisma ORM
|   └── shared-types/     # Interfaces TS compartidas
|
├── ai-services/
|   ├── fine-tuning/
|   └── prompts/
|
├── infra/
|   ├── nginx/
|   └── n8n/
|
├── docs/
|   └── ARQUITECTURA.md
|
├── docker-compose.yml
├── turbo.json
└── package.json
```

🧠 Filosofía del Monorepo

Ventaja	Beneficio
Código compartido	Tipos reutilizables
Build centralizado	Turbo cache
Deploy unificado	Docker Compose
Escalabilidad	Separación modular

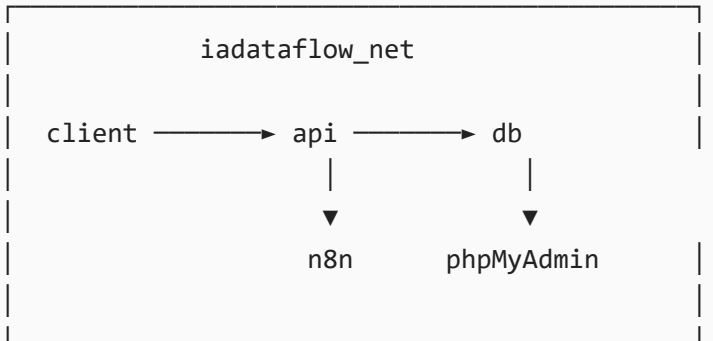
3 Stack Tecnológico

⚙️ Tecnologías Principales

Capa	Tecnología	Uso
Frontend	React 18	UI
Frontend	Vite	Bundler
Frontend	Tailwind CSS 4	Estilos
Frontend	shadcn/ui	Componentes
Frontend	React Router 7	Routing
Backend	NestJS 11	API REST
Backend	TypeScript 5	Tipado
DB	MySQL 8	Persistencia
ORM	Prisma	ORM
IA	LM Studio	LLM local
Automatización	n8n	Workflows
Infra	Docker	Contenedores
Monorepo	Turborepo	Orquestación

4 Servicios Docker

🐳 Red Docker





Servicios

Servicio	Puerto	Función
client	5173:80	Frontend
api	3000:3000	API REST
db	3307:3306	MySQL
phpmyadmin	8080:80	Admin DB
n8n	5678:5678	Automatización



Healthchecks

```
healthcheck:  
  test: ["CMD", "curl", "-f", "http://localhost"]
```

Warning

El contenedor `api` depende de que `db` esté healthy antes de iniciar.

5 Frontend — apps/client



Arquitectura Interna

```
src/  
├─ main.tsx  
├─ app/  
│   ├─ App.tsx  
│   ├─ routes.tsx  
│   ├─ components/  
│   └─ assets/  
└─ styles/
```

Contextos Globales

Contexto	Propósito
ThemeContext	Tema oscuro/claro
ProjectContext	Proyecto activo
NotificationContext	Notificaciones

Docker Build

Multi-stage Build

```
Stage 1 → Build React
Stage 2 → nginx runtime
```

Configuración nginx

Incluye:

- SPA fallback
- Cache estático
- Routing React

```
try_files $uri /index.html;
```

Backend — apps/api

Arquitectura NestJS

```
src/
├─ main.ts
├─ app.module.ts
├─ app.controller.ts
└─ app.service.ts
```

Variables de Entorno

Variable	Uso
DATABASE_URL	Conexión MySQL
JWT_SECRET	Tokens JWT
AI_ENGINE_URL	LM Studio
NODE_ENV	Ambiente

Docker Multi-stage

deps → builder → runtime

Seguridad

Important

La imagen final:

- NO contiene TypeScript
- NO contiene herramientas dev
- Usa usuario no-root (`nestjs`)

7 Base de Datos — `packages/database`

Prisma ORM

```
datasource db {
  provider = "mysql"
  url      = env("DATABASE_URL")
}
```

Migraciones

Crear migración

```
npx prisma migrate dev --name nueva_migracion
```

Producción

```
npx prisma migrate deploy
```

Prisma Studio

```
npx prisma studio
```

Tipos Compartidos — packages/shared-types

Interfaces Compartidas

```
interface ApiResponse<T> {  
  data: T;  
  message: string;  
  success: boolean;  
}  
  
interface PaginatedResponse<T> {  
  data: T[];  
  total: number;  
  page: number;  
  limit: number;  
}
```

Tip

Mantener contratos tipados entre frontend y backend evita inconsistencias.

Automatización — n8n

Motor de Automatización

Configuración	Valor
URL	http://localhost:5678
Persistencia	infra/n8n/data
Base interna	SQLite

Casos de Uso

- Webhooks
- Integraciones externas
- Automatización IA
- Emails
- Tareas programadas
- Slack / Discord / APIs

IA Local — ai-services

Recursos IA

```
ai-services/  
├─ fine-tuning/  
└─ prompts/
```

LM Studio

```
http://host.docker.internal:1234/v1
```

Info

`host.docker.internal` permite a Docker acceder al host Windows/Linux.

Infraestructura — `infra/nginx`

Reverse Proxy

Dominio	Destino
iadataflow.com	client
api.iadataflow.com	api
n8n.iadataflow.com	n8n

WebSockets

Configurado para:

- Tiempo real
- Streaming IA
- Eventos push

Variables de Entorno



`.env`

```
# Base de datos
DB_PASSWORD=mi_super_clave_123
DB_NAME=ia_dataflow

# Seguridad
JWT_SECRET=clave_larga_y_segura

# IA Local
AI_ENGINE_URL=http://host.docker.internal:1234/v1
```

```
# n8n
N8N_HOST=localhost
N8N_PROTOCOL=http
TIMEZONE=America/Bogota
```

Danger

Nunca subir `.env` al repositorio.

1 3 Flujo de Desarrollo

Desarrollo Local

Instalar dependencias

```
npm install
```

Levantar entorno

```
npm run dev
```

Turbo

```
npm run build
```

Turbo usa:

- Cache inteligente
 - Ejecución paralela
 - Dependencias automáticas
-

Docker

Primera vez

```
docker-compose up --build
```

Ejecución normal

```
docker-compose up
```

Logs

```
docker-compose logs -f api
```

Flujo de Producción

Recomendaciones

Buenas prácticas

- Pinear versiones
- SSL con Certbot
- Secretos seguros
- `restart: always`

Deploy Producción

```
docker-compose \  
-f docker-compose.yml \  
-f docker-compose.prod.yml \  
up -d
```

Pipeline CI/CD

```
Push a main
|
├─ Tests
├─ Build Docker
├─ Push Registry
└─ Deploy Producción
```

✓ Resumen Arquitectónico

Área	Tecnología
Frontend	React + Vite
Backend	NestJS
DB	MySQL + Prisma
IA	LM Studio
Automatización	n8n
Infraestructura	Docker + nginx
Monorepo	Turborepo

🧠 Principios del Proyecto

✓ Success

El proyecto sigue una arquitectura:

- Modular
 - Escalable
 - Docker-first
 - AI-ready
 - Monorepo-oriented
 - Production-ready
-